

docs_chain

कोई भी ट्रैक किया गया दस्तावेज़ सिंक से बाहर नहीं हो सकता — कंटेंट-हैश, द्विदिशात्मक, परमाण्विक।

सारांश

Docs Chain एक सार्वभौमिक, Go-आधारित द्विदिशात्मक दस्तावेज़-और-डेटाबेस निर्भरता-प्रसार इंजन है। जब किसी पंजीकृत श्रृंखला के किसी भी सदस्य में परिवर्तन होता है — मार्कडाउन स्रोत, HTML/PDF/DOCX निर्यात, या SQLite डेटाबेस — तो यह कंटेंट हैश के माध्यम से परिवर्तन का पता लगाता है और उसे परमाण्विक रूप से हर जुड़े सदस्य तक प्रसारित करता है।

संक्षिप्त विवरण

एक Go इंजन जो दस्तावेज़ों और डेटाबेस को सिंक में रखता है। साल्सा-शैली के कंटेंट-हैश वृद्धिशील पुनर्गणना का उपयोग करते हुए एक DAG (कान टोपोलॉजिकल ऑर्डरिंग, अर्ली कटऑफ़, द्विदिशात्मक सिंक एज, परमाण्विक-रीनेम + SQLite-लेनदेन कमिट्स) पर, यह किसी भी जुड़े आर्टिफैक्ट में परिवर्तन होने पर निर्यातों को पुनः उत्पन्न करता है।

विस्तृत विवरण

Docs Chain वही है जिसे आप तब बनाते हैं जब आपने एक ही नाजुक “मार्कडाउन बदलने पर PDF को पुनः उत्पन्न करें” शेल स्क्रिप्ट को एक बार ज्यादा लिख लिया हो। यह उस संपूर्ण हस्तनिर्मित सिंक गोंद को एक वास्तविक इंजन से बदल देता है। यह किसी प्रोजेक्ट के दस्तावेज़ों और डेटाबेस को एक श्रृंखला के सदस्य के रूप में मॉडल करता है और जब किसी सदस्य में परिवर्तन होता है, तो उसे हर घोषित दिशा में हर जुड़े सदस्य तक प्रसारित करता है — परमाण्विक रूप से पुनः उत्पन्न और निर्यात करता है ताकि कोई भी ट्रैक किया गया आर्टिफैक्ट कभी सिंक से बाहर न हो सके। इसका डिज़ाइन अपनी कठोरता सीधे वृद्धिशील बिल्ड सिस्टम की दुनिया से उधार लेता है, न कि स्क्रिप्टिंग से: परिवर्तन का पता **कंटेंट हैश** से लगाया जाता है, **mtime** से नहीं, इसलिए touch कुछ भी ट्रिगर नहीं करता और एक-बाइट का संपादन वही पुनर्निर्माण ट्रिगर करता है जो उसे करना चाहिए — न झूठी चेतावनी, न छूटा हुआ परिवर्तन। इसे औपचारिक रूप से एक पंक्ति में कहा जाए तो यह साल्सा-शैली की कंटेंट-हैश वृद्धिशील पुनर्गणना है एक DAG पर, जिसमें कान टोपोलॉजिकल ऑर्डरिंग, अर्ली कटऑफ़ जो अपरिवर्तित उपवृक्षों को काट देता है, घोषित-अधिकार द्विदिशात्मक sync एज, और परमाण्विक-रीनेम प्लस SQLite-लेनदेन कमिट्स हैं ताकि प्रसार के बीच में क्रेश होने पर कभी अधूरा निर्यात न रह जाए। यह vasic-digital सबमॉड्यूल के रूप में उपलब्ध है और HelixConstitution सबमॉड्यूल के

मुख्य भाग के रूप में उपयोग किया जाता है, इसलिए कोई भी प्रोजेक्ट जो संविधान को अपनाता है, उसे Docs Chain स्वतः प्राप्त हो जाता है और अपने संदर्भों के लिए YAML के माध्यम से अपनी श्रृंखलाएँ पंजीकृत करता है। कार्यान्वयन स्थिति के प्रति ईमानदार है (संविधान §11.4.6 के अनुसार): चरण 1-4 (कोर DAG + हैशिंग, नोड एडेप्टर/ट्रांसफॉर्म, परमाण्विकता के साथ प्रसार ऑर्केस्ट्रेटर, कॉन्फ़िग-चालित बहुसंदर्भ CLI जिसमें sync/verify/doctor/graph/watch) लागू और परीक्षित हैं; चरण-4b में सामान्य द्विदिशात्मक md-to-sqlite/sqlite-to-md बिल्ट-इन (शुद्ध-Go, पंक्ति-स्तरीय ड्रिफ्ट, बाइट-स्थिर राउंड-ट्रिप) और colorize-html बिल्ट-इन जोड़े जाते हैं; चरण 5 में व्यापक वास्तविक-बाइनरी e2e लागू और GREEN है। चरण 6-7 (संविधान वितरण, ATMOSphere वायरिंग) योजनाबद्ध हैं और ऑपरेटर-गेटेड रहेंगे। Herald पहला वास्तविक डाउनस्ट्रीम उपभोक्ता है, जो 66-दस्तावेज़ बहु-प्रारूप कॉर्पस को सिंक करता है जो स्वच्छता की पुष्टि करता है।

हमने इसे क्यों बनाया

दस्तावेज़ीकरण, निर्यात और डेटाबेस हाथ से या नाज़ुक स्क्रिप्ट से बनाए रखने पर तुरंत अलग हो जाते हैं। Docs Chain सिंक्रोनाइज़ेशन को यांत्रिक, कंटेंट-हैश-सटीक और परमाण्विक बनाता है, ताकि श्रृंखला में कहीं भी परिवर्तन होने पर सब कुछ डाउनस्ट्रीम (और अपस्ट्रीम) सही और सुरक्षित रूप से अपडेट हो जाए।

सामग्री

यह क्यों एक गेम-चेंजर है

यह उन कठिनाई से अर्जित शुद्धता की गारंटियों को, जिन्हें कंपाइलर और बिल्ट-सिस्टम के लेखक स्वाभाविक मानते हैं — कंटेंट-हैश डिपेंडेंसी ग्राफ़, न्यूनतम पुनर्गणना, एटॉमिक कमिट्स — दस्तावेज़ीकरण और डेटाबेस की दुनिया में लागू करता है, जो ऐतिहासिक रूप से क्रॉन जॉब्स और अच्छे इरादों पर ही निर्भर रहा है। सच्चा द्विदिशात्मक सिंक यह सुनिश्चित करता है कि स्रोत और उसके निर्यात के बीच का संबंध दोनों दिशाओं में लागू हो, जिससे “दस्तावेज़ पुराने हो गए हैं” और “निर्यात स्रोत से मेल नहीं खाता” जैसे बार-बार आने वाले बग समाप्त हो जाते हैं और ऐसी स्थितियाँ बन जाती हैं जिन्हें इंजन कभी उत्पन्न होने ही नहीं देता।

क्या है नवीन

- कंटेंट-हैश (न कि संशोधन समय) पर आधारित वृद्धिशील पुनर्गणना, जिसमें शीघ्र समाप्ति के साथ DAG का उपयोग।
- द्विदिशात्मक, घोषित-अधिकार सिंक किनारे (दस्तावेज़ ↔ निर्यात ↔ SQLite)।
- क्रैश-सुरक्षित प्रसार के लिए एटॉमिक-रीनेम + SQLite-लेनदेन कमिट।
- शुद्ध-Go md-to-sqlite/sqlite-to-md चक्र, जिसमें पंक्ति-स्तरीय विचलन पहचान।

चुनौतियाँ और समाधान

- अनावश्यक पुनर्निर्माण: कंटेंट-हैश पहचान द्वारा हल (समय-मुद्रा के बजाय)।
- आंशिक/भ्रष्ट अपडेट: एटॉमिक-रीनेम और SQLite लेनदेन द्वारा हल।

- सही बहु-सदस्यीय क्रम: काह्ल टोपोलॉजिकल क्रम + शीघ्र समाप्ति द्वारा हल।
- ईमानदार क्षमता रिपोर्टिंग: प्रत्येक चरण को §11.4.6 के अनुसार IMPLEMENTED या PLANNED के रूप में चिह्नित करके हल।

तकनीकी ढाँचा (क्यों + कैसे)

- **Go** — संपूर्ण इंजन (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain)।
- **DAG + काह्ल टोपोलॉजिकल सॉर्ट** — शीघ्र समाप्ति के साथ निर्भरता क्रम।
- **SQLite (शुद्ध-Go modernc)** — डेटाबेस सदस्य और लेनदेनात्मक कमिट।
- **fsnotify** — लाइव प्रसार के लिए watch डेमन।
- **YAML कॉन्फिग** — प्रति-संदर्भ श्रृंखला पंजीकरण।
- **exec: ट्रांसफॉर्म** — प्लगइन योग्य Markdown→HTML/PDF/DOCX जनरेशन।

रोडमैप की ईमानदारी: चरण 6-7 (संविधान वितरण, ATMOSphere एकीकरण) PLANNED / ऑपरेटर-नियंत्रित हैं — अभी तक जारी नहीं किए गए।